

Prob 1. (45 pts) Assume we are given the following code snippets:

Part A: C code:

```
1  // Prototype of functions defined in a .s file
2  extern void task_a(uint8_t *pIpt, int32_t *pOup);
3
4  // Global variable
5  uint8_t ipt[20];    // Input data
6  int32_t opt[4];     // Output data
7
8  int main(void) {
9      for (int i = 0; i < 20; i++) {
10         ipt[i] = 0x70 + (i<<2);
11     }
12
13     // Task 1.
14     task_a(ipt, opt);
15     printf("opt[0] = 0x%X, opt[1] = 0x%X, opt[2] = 0x%X, opt[3] = 0x%X\n",
16         opt[0], opt[1], opt[2], opt[3]);
17
18     while (1);
19 }
```

Part B: assembly code:

```
1  task_a:
2      ldrrd    r2, r3, [r0, #8]!
3      strd     r2, r3, [r1, #0]
4
5      ldrsb    r2, [r0, #4]!
6      ldrsb    r3, [r0, #4]!
7      strd     r2, r3, [r1, #8]
8
9      bx      lr
10
11  .end
```

Answer the following questions (without running the program first):

1. (6 pts) Determine the memory map after running the above C code.
2. (16 pts) Determine the printout after running `task_a`.
3. (8 pts) Determine values of R0 and R1 just before exiting `task_a`.
4. (15 pts) Translate the ASM function into a C function.

Prob 2. (20 pts) Given a C function shown below.

```
1  int math_fun_c(int x) {  
2      if (x < -5) {  
3          x = - 5;  
4      } else if (x > 5) {  
5          x = 5;  
6      }  
7      return x;  
8  }
```

Translate the above C function to ASM using the prototype below: `int math_fun_s(int x).`

Prob 3. (35 pts total) Write a C function (15 pts) and translate it to ASM (20 pts) to find the summation of negative elements of an array with N elements using prototypes `int mp_neg_sum_array_c(int *arr, int N)` and `int mp_neg_sum_array_s(int *arr, int N)`